AOC2 Project 1 - Convolutional Neural Network Unit in a MIPS-based SoC

Javier Resano, José Luis Briz, and Alejandro Valero

Dept. of Computing Science and Systems Engineering, EINA - Univ. Zaragoza

Submission deadline: 07-04-2026 07:55 a.m.
s.t. confirmation via Moodle.

README

- **Save time by carefully reading this guide.**
  - This guide effectively serves as an extended lecture on Computer Architecture and Organization, covering details that are difficult to fully address in the classroom.
  - It will help clarify key concepts that may still be unclear despite the theoretical material and problem sets.
  - It describes the steps you must follow, the specific actions you are expected to take, and provides guidance for organizing and writing your report.
- **Do not change component or signal names** in the provided VHDL source code without consulting the faculty beforehand.
- All materials (source files, slides, problem sets, guides, etc.) are subject to the Intellectual Property Rights of the University of Zaragoza. **Any violation of these rights may have legal consequences.**
- **Language note.** AOC2 is an ELF (*English Language Friendly*) course. The main reference books and most in-house materials are provided in English. As the course is still transitioning from a non-ELF format, you may encounter symbols and comments in Spanish within the provided VHDL source code. If you are not a native Spanish speaker and encounter difficulties, do not hesitate to ask the faculty for clarification.
- Please report any typos or errors you may find, and feel free to ask questions when clarification is genuinely required.
  - Developing the ability to extract and apply technical information from written documentation is an essential part of this course.
  - You are therefore expected to read this guide carefully before seeking assistance.
  - *Documented behavior should be consulted before assuming undefined behavior*.

☐ The checkboxes throughout this guide serve as a structured checklist to help you verify the key aspects of your project prior to submission and to prepare for the in-person interview.

Contents

1 Summary

The objective of this project is **to gain proficiency in working with a hardware description language** within a professional simulation environment using testbenches and chronograms. The case study is a 32-bit MIPS processor with a 5-stage pipeline, consistent with the architecture introduced in lectures and lab assignments, and boasting an ALU which supports a vector Multiply-Accumulate (MAC) operation (see

Sec. 2.1). This operation is a fundamental building block of Convolutional Neural Networks (CNNs).[1]

The principal steps in this project are as follows:

- ☐ Extend the baseline MIPS architecture to support the two instructions from Lab Assignment #3. This requires modifications to the Control Unit and the datapath (see Sec. 4.1).
- ☐ Implement data forwarding to eliminate data hazards where possible; otherwise, the processor must be stalled in the `IF` and `ID` stages (see Sec. 4.2). The destination of forwarded data will always be the `EX` stage.
- ☐ Address potential **control hazards** (see Sec. 4.3).
- ☐ Implement a **set of performance counters** to monitor system efficiency (see Sec. 2.4).
- ☐ The provided ALU executes a MAC operation in one cycle. You have to modify the ALU to implement a multi-cycle vector MAC, consequently treating the new structural hazard (see Sec. 2.1)
- ☐ Once the previous modifications are fully functional, you must verify the **management of exceptions** (elements, operation, and handling) as provided in the MIPS source code.
- ☐ Design testbenches and test programs to validate each step (unit testing) as well as the overall design (integration testing), as detailed in Sec. 5.1.
- ☐ Draft a report following the specifications provided in Sec. 5.2.

The project is designed to be completed in **pairs**. Larger groups are not permitted (see Sec. 6).

2 Baseline MIPS

The baseline MIPS provided as the starting point for this project implements the architecture discussed in lectures, offering more detail than the version used in the compulsory lab assignments. Its specific features are as follows:

- It implements a **fully delayed MIPS ISA**: the hardware does not manage hazards of any kind (i.e., the pipeline never stalls, and no resources are provided for data forwarding). Any code executed on this core *out-of-the-box* must avoid hazards through static instruction scheduling, such as reordering instructions or inserting *nops*.
- Register $r0$ functions as a standard general-purpose register.[2]
- Branches are resolved in the `ID` stage.
- Register writes are generally rising-edge triggered, with the exception of the register file (`RF`), which is **falling-edge triggered**.
- Includes support for exceptions (see Sec. 2.3).
- The ALU supports the execution of a MAC operation (instructions `mac_ini` and `mac`, see Sec. 2.1).
- Provided as a core integrated within a *System-on-Chip* (SoC, see file `AOC2_SoC.vhd`), alongside an IO data memory subsystem (see Sec. 2.2).

[1] You might want to visit *Building Blocks* section of this Wikipedia entry to find out how the MAC operation applies: https://en.wikipedia.org/wiki/Convolutional_neural_network.

[2] In the original MIPS ISA, a `lw` to $r0$ always returns 0, and a `sw` to $r0$ has no effect.

2.1 Vector MAC operation

The ALU in this project's MIPS processor supports the standard integer instructions `and`, `or`, `add`, and `sub`, as well as two vector multiply–accumulate (MAC) operations, namely `mac` and `mac_ini`, defined as follows:

```
mac rd, rs, rt:
    rd, ACC <=  ACC + rs[31 downto 24] * rt[31 downto 24] +
                rs[23 downto 16] * rt[23 downto 16] +
                rs[15 downto 8] * rt[15 downto 8] +
                rs[7 downto 0] * rt[7 downto 0];
    PC <= PC + 4;
mac_ini rd, rs, rt:
    rd, ACC <=  rs[31 downto 24] * rt[31 downto 24] +
                rs[23 downto 16] * rt[23 downto 16] +
                rs[15 downto 8] * rt[15 downto 8] +
                rs[7 downto 0] * rt[7 downto 0];
    PC <= PC + 4;
```

The `mac` and `mac_ini` instructions take `rs` and `rt` as vectors of four bytes each and use a shadow `ACC` (accumulator) register to perform the reduction, i.e., the final sum of all element-wise products. Both instructions assume signed operands. In Moodle you can find a detailed example of a neural network code that uses these two instructions assuming that each operand is a 8-bit real number in Q4.4 format, which means that 4 bits are used for the integer part and 4 bits for the fractional part.

- ☐ Inspect the provided source code and draw a block-level sketch of the vector unit executing these instructions.
- ☐ Check out Sec. 7.3 to better understand the way we deal with fixed-point representation and arithmetic through the MAC unit.

2.2 IO Data Memory Subsystem

The subsystem in `IO_MD_subsystem.vhd` comprises the MIPS data DRAM and several memory-mapped I/O registers, all accessible via the `lw` and `sw` instructions. Two of these registers are used for external data exchange, while a third is dedicated to interrupt management. Communication between the MIPS core and the I/O data memory subsystem is implemented through a set of buses and three primary control signals:

- `IRQ`: (see Sec. 2.3). This signal indicates an incoming interrupt request. The sole interrupt source in this system is the external signal `Ext_IRQ`, which `IO_MD_subsystem.vhd` forwards to the MIPS core as `MIPS_IRQ`. The core acknowledges the interrupt by writing `1` to the 1-bit internal register `ACK_REG`. This register automatically resets to zero in the following cycle to simplify the acknowledgment handshake. The status of `ACK_REG` is exposed via the output signal `INT_ACK`.
- `Data_Abort`: (see Sec. 2.3). This signal indicates an illegal memory access, such as an unaligned address or an out-of-range access. For debugging purposes, the address bus will reflect the value `0xBAD0ADD0` when this signal is asserted.
- `IO_Mem_ready`: this signal is asserted when the DRAM is ready to complete the requested access within the current cycle. If a memory instruction (`lw` or `sw`) reaches the `MEM` stage while `IO_MEM_ready` is deasserted (`0`), the control unit must stall the entire pipeline. To facilitate debugging, attempting to read the data bus while the DRAM is not ready will return the value `0x000BAD00`.

The IO data memory subsystem also includes two 32-bit registers (`IO_input` and `IO_Output`) used to interface values with the external environment.

Event	**Exception**	**Origin**	Vector @
Reset	*reset*	External (testbench)	0x0
Interrupt request	*irq*	IO memory subsystem (signal `IRQ` = 1)	0x4
Reference to an invalid data memory address	*data abort*	`Data_Memory_Subsystem` (signal `Data_abort` = 1)	0x8
Unknown opcode	*undef*	`UC`	0xC

Table 1: Exceptions in this project's MIPS core.

2.3 Support for Exceptions

Tab. 1 summarizes the four events that can trigger an *exception* in this project's MIPS core. The rightmost column specifies the corresponding interrupt vectors.[3]

The processor relies on the following hardware resources to manage these exceptions:

- **Status Register** (`status_reg`, 2 bits):
  - Bit 1 (MSB): setting this bit to 1 disables exceptions; clearing it to 0 enables them.
  - Bit 0 (LSB): this bit indicates the processor state; it is 0 during normal execution and 1 when the processor enters exception mode.
- **Exception Link Register** (`Exception_LR`): this register captures the return address, allowing the processor to resume the main program once the exception routine completes.
- **Return Address Multiplexer**: this mux selects the appropriate return address to maintain correct program order, choosing between the instructions currently in the `EX`, `ID`, or `IF` stages. A dedicated validity signal is propagated alongside each instruction through the pipeline; additionally, the register banks for the `ID` and `EX` stages are equipped with registers to store the Program Counter (PC) of the instruction currently occupying that stage.
- **Extended PC Multiplexer**: the `PC` input mux includes additional entries to load either the address of the required Exception Service Routine (sourced from the corresponding exception vector) or the return address stored in the `Exception_LR_output`.

The MIPS core executes the following sequence of actions when exceptions are enabled and one of the events listed in Tab. 1 is triggered:

1. Instructions currently in the `MEM` and `WB` stages complete their execution normally. Conversely, instructions in the `IF`, `ID`, and `EX` stages are cleared (invalidated). You should study the provided source code to understand how this flush is implemented.
2. The address of the oldest valid instruction being flushed from the pipeline is stored in the register `Exception_LR`, enabling correct program resumption.
3. The address of the corresponding Exception Service Routine (ESR) is loaded into the `PC` from the appropriate offset in the exception vector table. E.g.,in the event of an `IRQ`, the processor performs: `PC <= x"00000004"`.

[3] Memory addresses that store the starting address of the Exception Service Routine (ESR) to which the processor branches when a specific exception occurs.

4. The status register switches to exception mode, disabling all exceptions (`Status <= "11"`).

Note that the only exception after which program resumption makes sense is `IRQ`, via execution of the `rte` instruction (see Sec. 4.1).[4]

2.4 Performance Counters

To analyze a program's performance on a given microprocessor, we rely on tools known as *profilers*[5]. Modern profilers are typically based on hardware event counters integrated into the processor. In this project, the MIPS core must support the following event counters:

- Cycles since last reset.
- Dynamic instruction count: valid instructions completing in `WB`.
- Stall cycles due to data hazards.
- Stall cycles due to control hazards.
- Stall cycles due to multi-cycle memory references.
- Stall cycles due to structural hazards generated by the multi-cycle MAC operations.
- Total number of accepted exceptions.

3 Your preliminary ToDo list

☐ Download the companion materials from Moodle (source files, test, examples).

☐ Study the source code to understand the structure and function of the SoC and its components (MIPS and IO Memory subsystem). Identify the components introduced in the theory classes, problems, and Lab Assignment #3.

- *Scratching your own schematics of the SoC and the MIPS will be definitely useful to you. Write down the names of the principal signals connecting the main components.*

☐ Create a project in the VHDL simulation environment you used in Lab Assignment #3. Compile and simulate the example test. Organize your own waveform of the MIPS and save it. Check out how the signals you see in your waveform evolve, according to instructions in the example test.

- ☐ Track signal propagation through the register banks of consecutive stages.
- ☐ When exceptions are enabled pay special attention to how exceptions work (control transfer, mode switching, `rte` instruction). Verify that the test bench provided for exceptions is working properly.
- ☐ Check out the values written in the register file, data memory, and IO_ouput.

[4] ESRs for *data abort* and *undef* should halt the processor (as in our tests) or pass control to the operating system, using `Exception_LR` to report the triggering error.

[5] A widely used example is *perf*.

4 Tasks

The provided source code describes a baseline MIPS implementation that does not manage hazards and never stalls the pipeline (see Sec. 2). Consequently, it is unable to correctly execute programs compiled for a non-delayed ISA.[6] You must implement the necessary modifications to support a non-delayed MIPS ISA. This involves detecting structural, data, and control hazards, implementing data forwarding where possible, and stalling the pipeline when necessary. The updated design must correctly execute the three new instructions.

4.1 Adding New Instructions

Design and apply the required modifications to support the following instructions:

- ☐ `jal` and `ret`, following the same semantics as in Lab Assignment #3.

4.2 Managing Structural and Data Hazards

You must complete the following two components:

- ☐ **Forwarding Unit** instantiated as `UA`[7]. `UA` controls the two multiplexers at the ALU inputs in the baseline MIPS source code. You must design the logic that selects operand values for a consumer instruction in the `EX` stage, sourcing them either from the `RF` or from the forwarding buses. Pay particular attention to the operands of the new instructions including `mac` and `mac_ini`. Consider, for instance, whether a value generated by `jal` in `WB` can be forwarded to a consumer at distance 1 through existing paths, and whether any further hardware support is justified.
- ☐ **Hazard Detection Unit** instantiated as `UD`[8].`UD` must detect data hazards that cannot be resolved through forwarding, such as load–use cases, or hazards involving instructions that read operands in the `ID` stage (e.g., `beq` or `ret`). When such a hazard is detected, the `stall_ID` signal must be asserted to stall the `ID` and `IF` stages until the hazard is resolved. Stalling `ID` requires invalidating the instruction forwarded to the `EX` stage.

  In addition to data hazards, the `UD` also handles structural hazards arising from the `mac`/`mac_ini` unit and from memory stalls. If either the MAC unit or the memory subsystem is busy, the unit must assert `stall_MIPS`, which freezes all pipeline stages until both the MAC unit and the I/O memory subsystem are ready (see `IO_MEM_ready` in Sec. 2.2). You will revisit this mechanism after completing the multicycle Vector MAC unit (Sec. 4.4).

  Note that `stall_MIPS` also disables exception handling. In contrast, `stall_ID` does not interfere with exceptions, since branching to an exception service routine inherently nixes the instruction currently in `ID`

Watch out

- ☐ **Mind the validity bit** when enabling forwarding or stalling stages. Logic for forwarding data or stalling the pipeline should only trigger for valid instructions; performing these actions for invalid instructions is a logical error.

[6] In the baseline version, producer and consumer instructions must be separated by independent instructions (`nop` if none are available). Likewise, `beq`, `jal`, and `ret` require static scheduling, with `nop` instructions inserted as needed.

[7] *Unidad de Anticipación*

[8] *Unidad de Detención*

- ☐ **Stall only upon a hazard**. Precisely identify which operands (`rs` and `rt`) are required by each instruction and which instructions write to the `RF`, including the newly implemented ones. Note the following:
  - `nop` instructions do not generate data dependencies; consequently, they cannot cause data hazards. Stalling on a `nop` will be considered a minor mistake.
  - A `sw` instruction uses `rt` as a source (the data to be stored) and does not write to the `RF`. As it is not a producer, it cannot cause load-use hazards. Incorrectly stalling on a `sw` will be considered a minor mistake.
- ☐ Check and understand the template code provided in the `UD` for asserting operand usage and instruction validity. You must extend these assertions to include the logic and cases arising from the new instructions. You must use the signals defined in the baseline `UD` to partially or entirely stall the pipeline.

4.3 Managing Control Hazards

All control transfers in the provided baseline MIPS are delayed, because the processor does not manage any hazards. Now, you must modify the `UD` to support the management of control hazards. There are four control transfer instructions (`beq`, `jal`, `ret` and, `rte`). While these instructions are in the `ID` stage, the instruction in `PC+4` is being fetched from memory in `IF`.

- ☐ `jal`, `ret`, and `rte` are unconditional jumps, and therefore the instruction in `IF` must always be invalidated (clearing `valid_I_IF`).
- ☐ In the `beq` instruction, the instruction in `IF` is only invalidated if the branch is taken (T).[9]

4.4 Multicycle implementation of the ALU's vector MAC unit

At this point, you should have already understood the design of the default vector MAC unit included in this project's ALU source (see Sec. 2.1). The baseline vector MAC unit executes each `mac` instruction in a single cycle, potentially increasing the clock period T_c. To reduce this impact:

- ☐ Redesign the unit as a multicycle implementation by registering the four intermediate products (1st cycle), the partial product summation (2nd cycle), and the accumulation (3rd cycle), controlling it with an internal state machine. Follow the same approach we took in the pen-and-paper VHDL MAC design you solved in class.
- ☐ Revise your modified `UD` to deal with the new possible dependencies and answer the following questions: a) Which stages must be stalled? b) Which signal can you leverage to stall those stages? Mind that can only employ signals as defined in the provided`UD` (4.2).
- ☐ This MAC implementation requires a specific ABI[10]. The vector MAC unit includes architectural state beyond the general-purpose register file (i.e., the shadow `ACC` register). Consider a `mac` instruction inside a loop. If an `IRQ` occurs and the corresponding exception service routine (ESR) executes a `mac` instruction (or calls a library function that uses `mac`) analyze the effect on the interrupted loop once execution resumes. Identify the underlying issue and propose an appropriate architectural or software-level solution

[9] This is tantamount to say that we are predicting `beq` as not taken (NT).
[10] Application Binary Interface, a concept slipped in AOC1

4.5 Managing Performance Counters

Performance counters are disabled by default, except for the total cycle counter.

- ☐ Assign the proper value to their enabling signals so that they correctly record their corresponding events.
- ☐ Upon stalls, you must only enable one of the counters. For example, if there is a data hazard which implies a stall, and there is also a memory stall lasting four cycles, these four cycles add up to the memory stall counter, not to the data stalls counter. Once the memory becomes ready, we will increment the data stall counter if the data hazard persists.

5 Results and Deliverables

5.1 Testing

- Testing is what distinguishes an engineering project from a fortunate accident.
- This project can be tested in two complementary ways: through testbenches and through test programs. A basic testbench is provided to streamline your work. However, you are responsible for designing and executing both unit tests (targeting specific functionalities and corner cases) and integration tests to verify that all features specified in Sec. 4 operate correctly.

  > – The quality of your tests is part of the evaluation. Treat testing as a first-class component of the project.
  > – Passing tests is necessary; understanding them is required.

- When creating new programs or modifying existing ones, instructions must be encoded in hexadecimal and written into the instruction RAM component. The data RAM must also be updated if new or modified variables or constants are introduced.

Testing ToDo List

- ☐ Place your test programs inside a single `.vhd` file for all the instruction RAMs, as we do in the instruction RAM file we provide (`RAM_I_test_exceptions_neuron_2026.vhd`). Check that only one instruction RAM component remains un-commented (active) before compiling the project for simulating each test. Do not forget to make changes in the data RAM if required.
- ☐ Un-comment/comment the proper `stim_proc` as indicated in the source comments of the provided testbench, in order to check that your new instructions work correctly in the absence of interrupts. You can start leveraging the test program you used for Lab Assignment # 3, where your processor managed no hazards, with specific modifications as unitary tests.
- ☐ Go through all the test programs we include in (`RAM_I_test_exceptions_neuron_2026.vhd`) until all your modifications pass all of them.
- ☐ Simulate the test program delivered with the sources, activating IRQs in the testbench (using the `stim_proc` which triggers irqs in the testbench).
- ☐ Simulate your test programs to fine-tune on every potential problem you might spot.

5.2 Project Report

- ☐ Check that all sections specified in the report template provided with the companion materials have been properly completed. You may use the template *as is* or adapt it to your preferred format, but mind that all original sections are mandatory.

5.3 Estimated Project Timeline

1. Preliminary ToDo list (see Sec. 3): 2 h.
2. Adding the new instructions: 1 h.
3. Modifying the vector MAC unit: 1h.
4. Hazard management: 2 h.
5. Testing and debugging: 10 h.
6. Project Report: 3 h.

This estimate assumes you have kept up with the course (classes, problems, labs...), reasonably understand the concepts, and have thoroughly read this guide.

- ☐ Track the time you devote to the project. A simple spreadsheet can do the job.
- ☐ Compare the result with our estimate and write it down into the corresponding section of your report.

6 Grading Policy

We download, verify, and simulate each submission using our own evaluation test and classify projects into three categories:

S1: The report is satisfactory and the project passes our simulation test.

S2: The project fails our simulation test but passes the student-designed tests, which are soundly conceived and clearly justified, and whose issues can be reasonably corrected.

S3: None of the above.

Individual in-person interviews are then scheduled. During the interview, we may ask about any aspect of the project: report and this guide contents, function of signals, components, or VHDL constructs, behavior of specific instruction sequences (e.g., via a timing diagram).

Although the project is developed in pairs, interviews are conducted individually. Each student is expected to demonstrate a comprehensive understanding of all aspects of the project.

A clear and well-structured report significantly facilitates the interview.

Failure to demonstrate sufficient understanding during the individual assessment may result in a failing grade, even for projects initially classified as S1. Experience shows that this is rare: you generally perform well and demonstrate a good command of your work.

Projects classified as S2 that pass the interview may correct the identified issues and resubmit within 24 h.

7 Final Remarks

7.1 Instruction Format

> Instruction formats must strictly follow those defined in the baseline MIPS (see Unit 2 slides) and in this guide. Deviations will result in a failing project.

Please find a spreadsheet in the companion materials which will help you codify the instructions.

- Operation codes (opcode field):
  - `NOP: 000000; Arit: 000001; LW: 000010; SW: 000011`
  - `BEQ: 000100; JAL: 000101; RET: 000110; RTE: 001000`
- Function codes for ALU operations (*funct* field and signal `ALU_ctrl`):
  - `+: 00000; -: 00001; AND:00010; OR: 00011`

7.2 Memory addressing

Data and instruction memories (`RAM_D_test.vhd`, `RAM_I_test.vhd`) are defined in VHDL signals of type `array` with 128 words of 32 bits.

Each word in the VHDL RAM is a 32-b word codified in hexa (8 nibbles, e.g. `X"88010000", X"00000000"`…). Memory addresses are 32-bit wide, but we only consider 7 bits to address the RAM as an array of 128 words. Read thoroughly the VHDL code and find the relationship between memory address and word number in the array, so that you can easily convert back and forth effective address in a `lw`/ `sw` instruction and the word number in the VHDL array (which is the one you will see at the simulator's waveform).

7.3 Vector MAC fixed-point arithmetic

We are assuming fixed-point data representation for the input data (each input register byte). Key points to consider here are:

1. Multiplication Doubles Everything. When you multiply two $Q4.4$ numbers, the hardware performs an integer $8 \times 8 = 16$-bit multiplication. Mathematically: $(A \times 2^{-4}) \times (B \times 2^{-4}) = (A \times B) \times 2^{-8}$. This is why the products (`prod0_reg`, etc.) are interpreted as $Q8.8$. The fractional part doubled in size.
2. Addition Requires Alignment. You can only add numbers if their binary points are perfectly lined up. Since all four products are $Q8.8$, you can sum them directly. Since the Accumulator is also treated as having 8 fractional bits ($Q24.8$), you can add the sum of products to the ACC_out without any shifting.
3. The *Headroom* Concept. A CNN filter might involve summing hundreds of products. An 8-bit integer overflows quickly. By extending the sum to 32 bits ($Q24.8$), you are providing 16 bits of *Headroom* (2^{16} or $65,536$ times the range of a single product). This ensures that even with a large filter and high-magnitude weights, the accumulator won't overflow.

Table 2 will help you understand the way we are dealing with these three issues.

In your testbench, you can test the unit and your modifications codifying some real numbers into a $Q4.4$ format using a code like this one:

Stage	Bit Width	Binary Point	Format	Range
Inputs (DA/DB)	8-bit	Last 4 bits	Q4.4	-8.0 to $+7.9375$
Multiplier	16-bit	Last 8 bits	Q8.8	-128.0 to $+127.996$
Adder Tree	18-bit	Last 8 bits	Q10.8	Larger integer range
Accumulator	32-bit	Last 8 bits	Q24.8	Massive integer range

Table 2: Fixed-point arithmetic progression in vector MAC unit.

```
1  -- Conversion: Real_Value * 2^4 (for Q4.4)
2  -- Example: 1.5 * 16 = 24 (0x18)
3  -- Example: -2.0 * 16 = -32 (0xE0 in 8-bit signed)
4
5  DA <= "00011000"; -- 1.5  (Q4.4)
6  DB <= "00100100"; -- 2.25 (Q4.4)
7  -- Expected Product: 1.5 * 2.25 = 3.375
```

- We are referencing words, not bytes. Addressing an unaligned byte (i.e., a byte in a memory address which is not a multiple of 4) constitutes an invalid access and triggers a *data abort* exception (see Sec. 2.3).

8 Looking for Help

- Each project must be fully developed by the corresponding pair. Reusing code from others on a (near-) copy–paste basis is considered plagiarism. This does not preclude collaboration: you are encouraged to exchange ideas, discuss approaches, and help your classmates understand the problems.
- Do not hesitate to ask us for help. However, please note that the time we devote to individual projects is not formally accounted for. To make this process effective for everyone:
  - Be as specific as possible when contacting us by email. We can only process short, concise issues by email.
  - Consult the Moodle link *Project troubleshooting*, which is updated continuously throughout the course.
  - We strongly encourage requesting office hours (online or in person). This allows us to better understand your work and provide targeted feedback that will ultimately benefit your project and your grade.
  - Please keep in mind that our availability decreases non-linearly as the project deadline approaches.